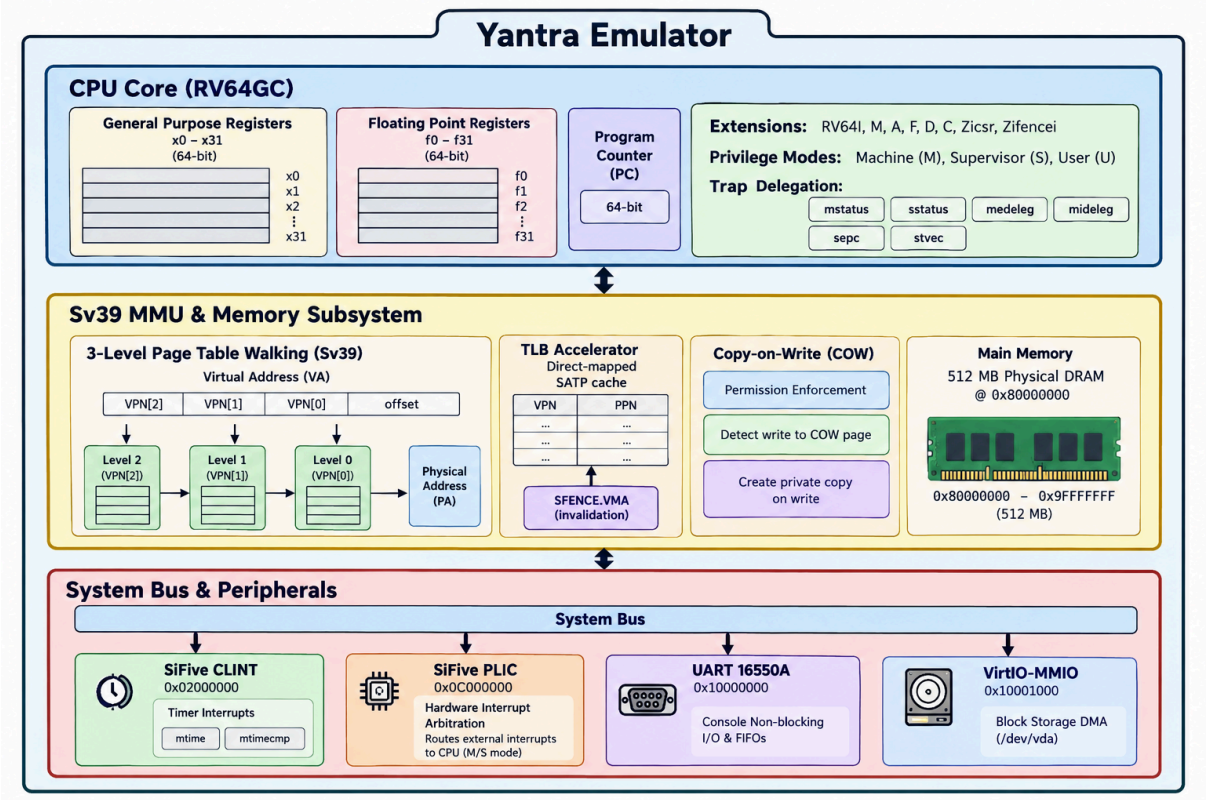
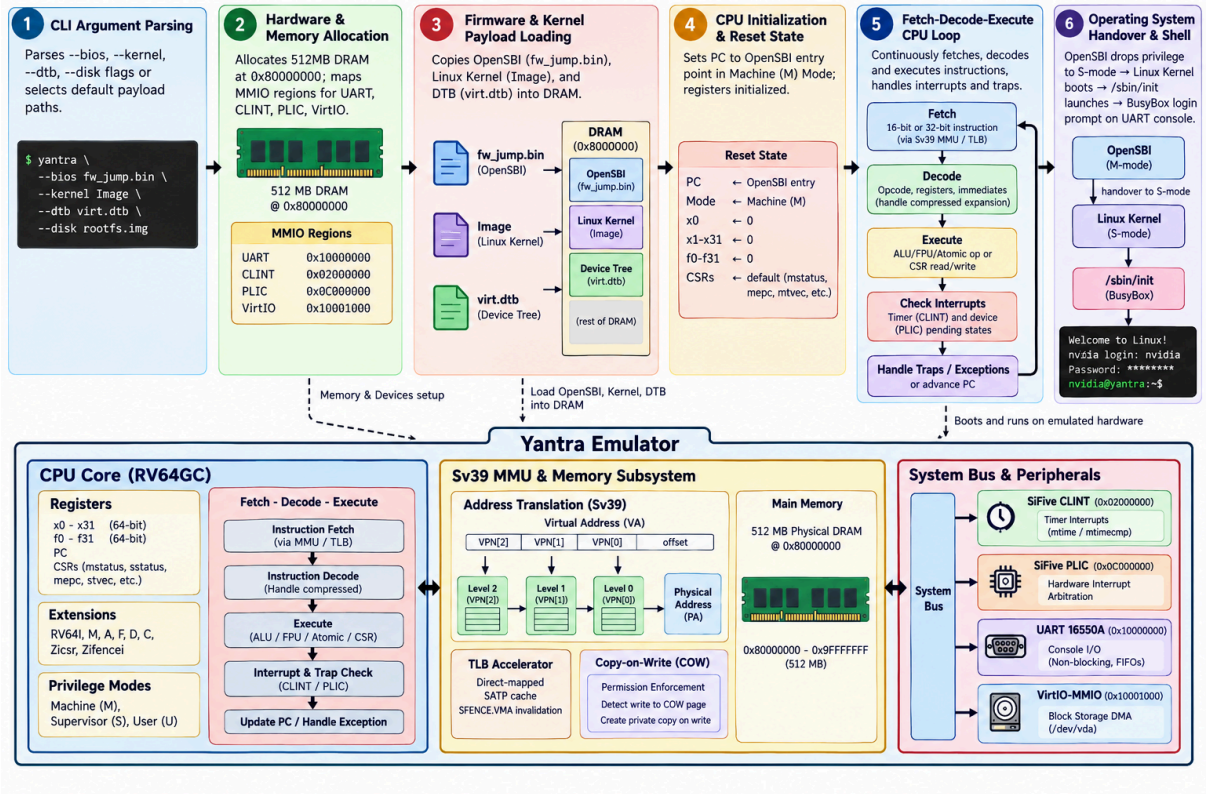
Architecture:



Execution flow:



Module 1: RV64I Base Integer ISA & CPU Core State

1. Executive Summary & Overview

The **RV64I** base integer instruction set forms the mandatory, fundamental layer of the 64-bit RISC-V architecture. In the **Yantra** system emulator, the CPU core maintains execution state using 32 general-purpose registers (`x0` through `x31`) alongside a 64-bit Program Counter (`PC`). Every 32-bit instruction fetched from physical memory is processed through a deterministic **Fetch-Decode-Execute** state machine that manages ALU computation, register file mutation, and PC advancement.

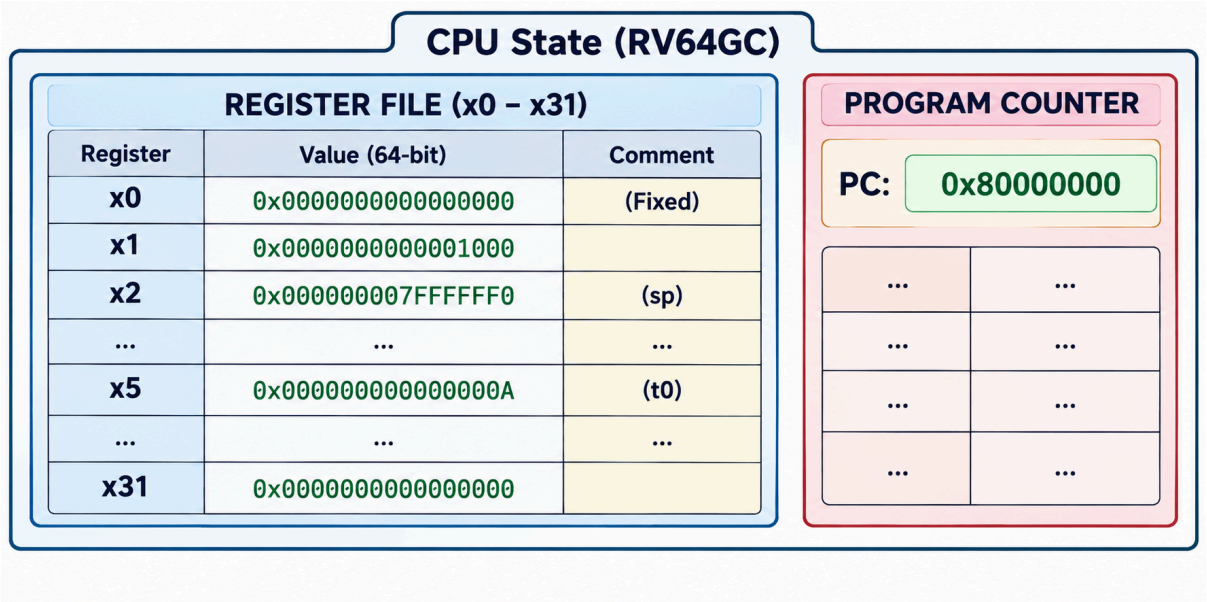
---

2. Core CPU State & Register File Architecture

In RV64I, register width ($XLEN$) is strictly 64 bits (`uint64_t`).

Register Index	ABI Name	Description / Saved State Rule	Hardware Constraint
`x0`	`zero`	Hardwired Zero	Immutable `0x0000000000000000`
`x1`	`ra`	Return Address	Caller-saved
`x2`	`sp`	Stack Pointer	Callee-saved
`x3`	`gp`	Global Pointer	Set on process startup
`x4`	`tp`	Thread Pointer	Thread-local storage
`x5`–`x7`	`t0`–`t2`	Temporary Registers	Caller-saved
`x8` / **`x9`**	`s0`/`fp`, `s1`	Saved Register / Frame Pointer	Callee-saved

`x10–x17`	`a0–a7`	Function Arguments / Return Values	Caller-saved
`x18–x27`	`s2–s11`	Saved Registers	Callee-saved
`x28–x31`	`t3–t6`	Temporary Registers	Caller-saved
`PC`	—	64-bit Program Counter	4-byte aligned (2-byte with RVC)



---

3. Register Access Mechanics & Hardwired Zero (`x0`)

3.1 Architectural Purpose

Register `x0` is hardwired to constant zero at the hardware level. This eliminates the need for separate dedicated instructions for common operations:

- **`NOP`** (No Operation) → `addi x0, x0, 0`
- **`MV rd, rs`** (Move) → `addi rd, rs, 0`
- **`NEG rd, rs`** (Negate) → `sub rd, x0, rs`
- **`SETEZ rd, rs`** (Set if Equal to Zero) → `sltiu rd, rs, 1`

3.2 Software Enforcement in Yantra

To ensure that writes directed to `x0` are safely discarded and reads always return zero, register access is guarded via helper functions:

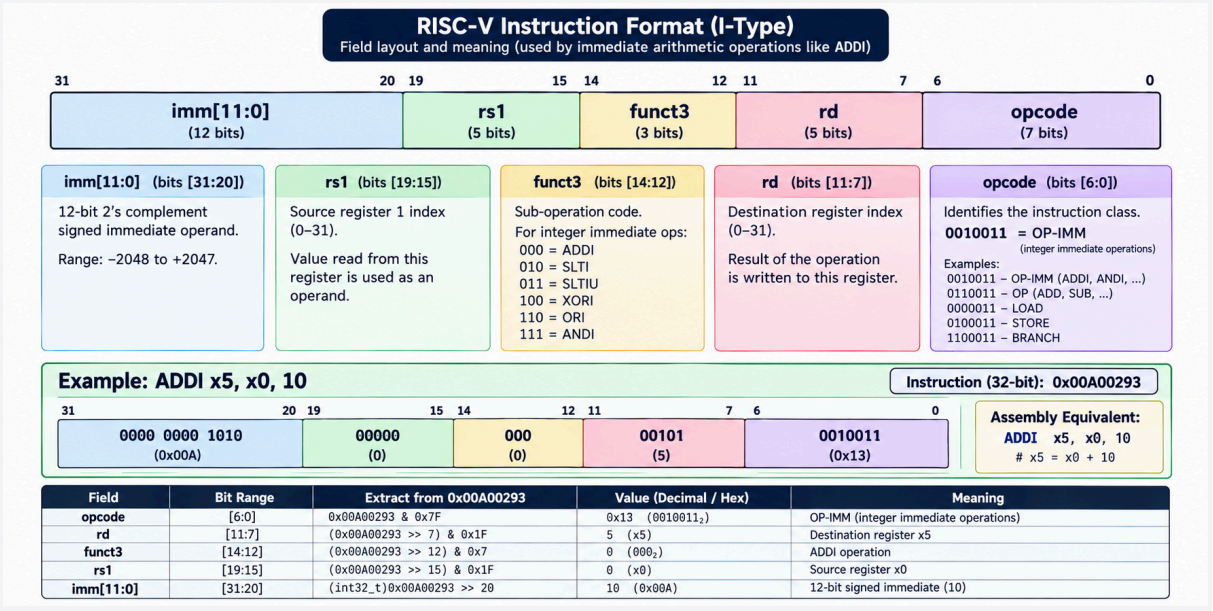
```cpp
// Reads always return 0 for x0
uint64_t read_reg(uint8_t reg) const {
    return (reg == 0) ? 0 : regs[reg];
}

// Writes targeting x0 are explicitly ignored
void write_reg(uint8_t reg, uint64_t val) {
    if (reg != 0) {
        regs[reg] = val;
    }
}
```

---

4. I-Type Instruction Format & Decoding

The `I-type` instruction layout packs a 12-bit signed immediate operand, source register index, destination register index, function code, and opcode into a fixed 32-bit word:



Deconstruction of Example Instruction (`0xFFF00293`):

- **Raw Word:** `0xFFF00293` (`111111111111 00000 000 00101 0010011_2`)
- **`opcode`** (`inst[6:0]`): `0x13` (`0010011_2` → `OP-IMM`)
- **`rd`** (`inst[11:7]`): `5` (`00101_2` → Register `x5` / `t0`)
- **`funct3`** (`inst[14:12]`): `0` (`000_2` → `ADDI` operation)
- **`rs1`** (`inst[15:19]`): `0` (`00000_2` → Register `x0`)

- **`imm[11:0]`** (`inst[31:20]`): `0xFFF` (12-bit 2's complement representation of `-1`)

---

5. Sign-Extension Mechanics in RV64

In 64-bit architecture, 12-bit immediates must be arithmetic sign-extended to 64-bit values prior to ALU evaluation.

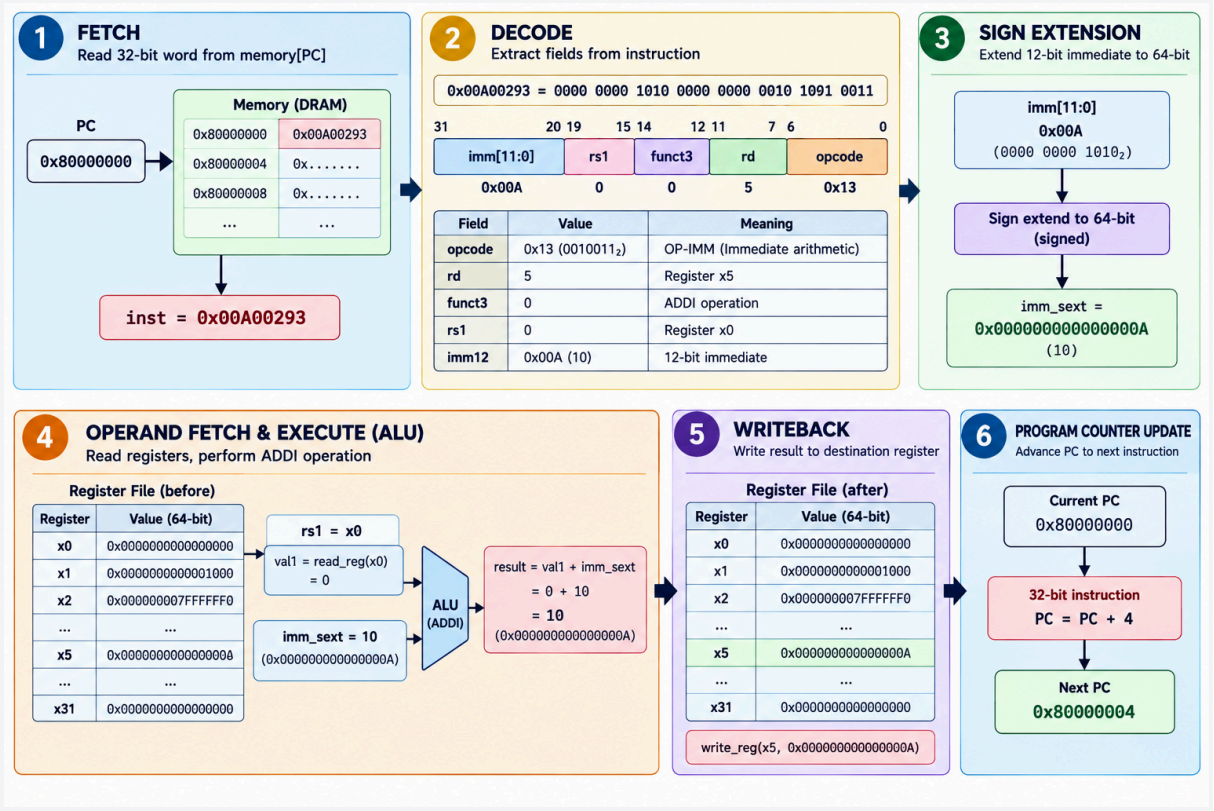
1. **Raw Extract:** `imm12 = (inst >> 20) & 0xFFF` → `0xFFF`
2. **Signed Cast:** Casting the 32-bit field `(int32_t)inst >> 20` preserves the sign bit (bit 31 of original instruction word).
3. **64-bit Widen:** Widening `(int32_t)(-1)` to `uint64_t` fills top 52 bits with `1`s: $$\text{imm_sext} = \text{0xFFFFFFFFFFFFFFFF}*16 \quad (-1*{10})$$

> **Critical Guardrail:** Zero-extending `0xFFF` would produce `0x0000000000000FFF` (`+4095`), which would corrupt address offsets and arithmetic calculations.

---

6. Step-by-Step Execution Trace (`addi x5, x0, 10`)

Given initial state: `PC = 0x80000000`, `regs[x5] = 0x0000000000000000`



---

7. Edge Cases & Implementation Checklist

- [x] **`x0` Target Writes:** Instructions targeting `x0` execute normally in ALU, but target writeback is discarded.
- [x] **Sign Extension:** All I-type, S-type, B-type, and J-type immediates must undergo sign-extension before computation.
- [x] **Word Operations (RV64I `ADDIW` / `ADDW`):** 32-bit word operations truncate results to 32 bits and sign-extend bit 31 through bit 63.
- [x] **PC Alignment:** PC increment must handle standard 4-byte boundaries and 2-byte boundaries when the C extension (RVC) is present.